

Astalakshimi Matrimony — High-Level Design (HLD)

Document version: 1.0 · Date: 2026-08-19 · Status: Based on actual codebase analysis

1. Executive Summary

Astalakshimi is a South Indian matrimony platform built as a monorepo using a modern TypeScript full-stack architecture. The system consists of a **Next.js 16 frontend** (App Router, React 19) and a **NestJS 11 backend API**, sharing types and validation schemas through internal workspace packages. Data is persisted in **PostgreSQL** via **Drizzle ORM**, and media files (photos, verification documents, horoscopes) are stored in **AWS S3** with presigned URL-based uploads.

Current State

Layer	Status
Frontend (Next.js)	✅ Fully implemented — landing, auth, multi-step registration, dashboard, discover/search, interests/inbox, profile view/edit, plans/checkout, notifications, settings, shortlist
Backend API (NestJS)	⚠️ Partially implemented — Auth, Profiles, Preferences, Media, Health modules are live. Search, Interests, Messaging, Notifications, Payments, Subscriptions, Admin, Moderation, Jobs, Shortlists are scaffolded but empty (0-byte files)
Database (PostgreSQL + Drizzle)	✅ Fully implemented — 8 tables with migrations, indexes, enums
Shared packages	✅ Types, Validation (Zod), Database client — all implemented
Infrastructure	⚠️ Docker/docker-compose directories exist but are empty (<code>.gitkeep</code> only)

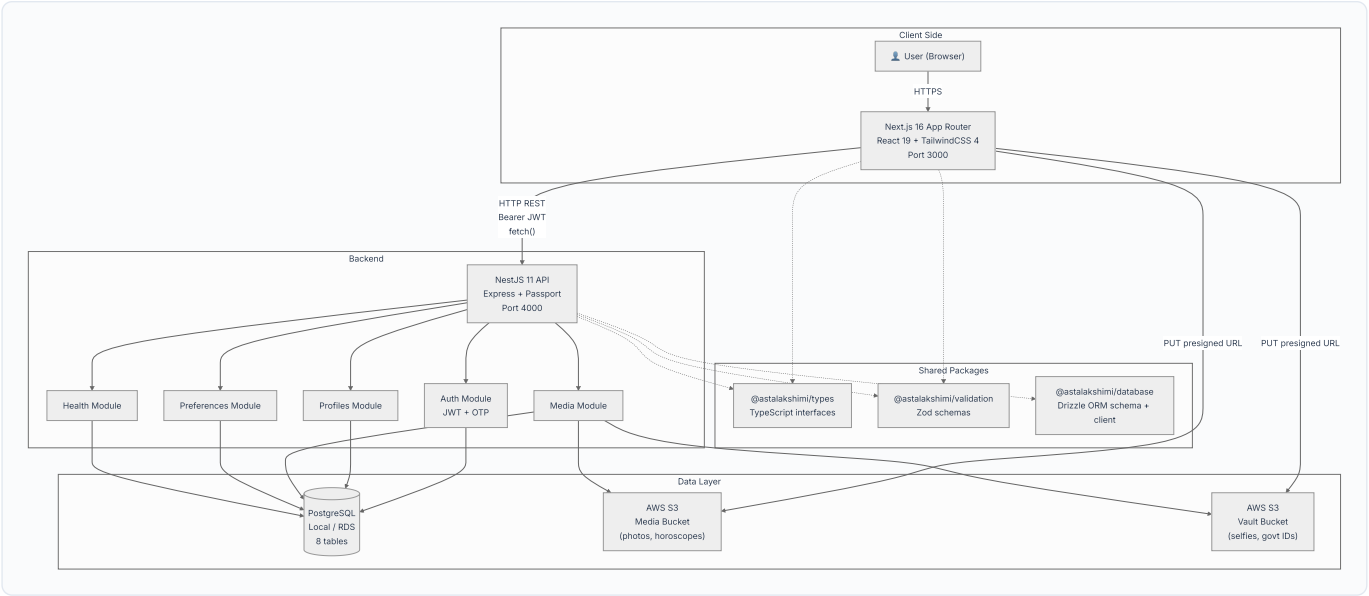
[!IMPORTANT] Many frontend features (interests, messaging, notifications, search/discover, shortlists, plans/payments, settings) currently operate on **client-side mock data** stored in `localStorage` / `sessionStorage` . Backend API endpoints for these features are **scaffolded but not implemented**. This document clearly distinguishes implemented from scaffolded/mock components.

2. System Overview

Astalakshimi is a matrimony platform targeting South Indian families, offering:

- **OTP-based phone authentication** (no password)
- **Multi-step profile registration** (6 steps: Auth → Identity → Community → Career/Family → Lifestyle/Astrology/Preferences → Photos/Verification)
- **Profile discovery** with filtering by age, community, city, religion, education, income
- **Interest sending & management** (send, accept, decline, withdraw)
- **Membership tiers** (Free, Silver, Gold, Platinum, Diamond)
- **Photo privacy** controls (blurred, visible after interest accepted, always visible)
- **ID verification** (selfie or government ID upload with 12-hour SLA)
- **Horoscope** upload and astrology matching
- **Presigned S3 uploads** for all media (photos → public bucket, verification docs → private vault bucket)

3. Architecture Diagram



4. Component Responsibilities

Component	Technology	Responsibility
apps/web	Next.js 16, React 19, TailwindCSS 4, Radix UI, Framer Motion, TanStack React Query	Frontend UI — SSR landing, client-side dashboard, form validation, S3 upload orchestration
apps/api	NestJS 11, Passport + JWT, Drizzle ORM, AWS SDK v3, Zod	REST API — Authentication, profile CRUD, media presigned URLs, partner preferences
packages/database	Drizzle ORM, postgres.js driver	Schema definitions, database client factory, migrations
packages/types	TypeScript	Shared interfaces: <code>User</code> , <code>Profile</code> , <code>FullProfileView</code> , <code>AuthResponse</code> , media types
packages/validation	Zod	Shared validation schemas: <code>sendOtpSchema</code> , <code>verifyOtpSchema</code> , <code>presignedUploadSchema</code> , <code>partnerPreferencesSchema</code>
packages/config	—	Reserved, empty (<code>.gitkeep</code>)
packages/api-client	—	Reserved, empty (<code>.gitkeep</code>)

5. Frontend Architecture

5.1 Technology Stack

Concern	Technology	Version
Framework	Next.js (App Router)	16.3.1
UI Library	React	19.2.8
Styling	TailwindCSS + custom CSS variables	4.x
Component Library	Radix UI (via <code>radix-ui</code>) + shadcn-style wrappers	1.6.7
State Management	TanStack React Query + <code>sessionStorage</code> + React state	5.x
Forms	React Hook Form + <code>@hookform/resolvers</code> (Zod)	7.85.0
Animations	Framer Motion	13.1.0
Icons	Lucide React	1.31.0
Validation	Zod (local + shared <code>@astalakshimi/validation</code>)	3.24.2
HTTP Client	Native <code>fetch</code> via custom <code>ApiClient</code> class	—

5.2 Route Structure (App Router)

Route Group	Path	Purpose	Layout
(public)	/	Landing page (hero, features, testimonials)	Root layout
(auth)	/login , /register	Phone OTP login, multi-step registration wizard	Root layout
(dashboard)	/home	Authenticated home feed	DashboardShell
(dashboard)	/dashboard	Discover — browse & filter matches	DashboardShell
(dashboard)	/search	Search with advanced filters	DashboardShell
(dashboard)	/interests	Sent/received interests, mutual matches	DashboardShell
(dashboard)	/plans , /checkout	Membership plans, payment flow	DashboardShell
(dashboard)	/profile , /profile/edit	My profile view & edit	DashboardShell
(dashboard)	/settings	Privacy & notification settings	DashboardShell
(dashboard)	/shortlist	Saved/shortlisted profiles	DashboardShell
(dashboard)	/notifications	Activity feed with deep links	DashboardShell
profiles	/profiles/[profileId]	View another member's profile	DashboardShell

5.3 API Client (`api-client.ts`)

A singleton `ApiClient` class wrapping native `fetch` :

- **Base URL:** `NEXT_PUBLIC_API_URL` (default: `http://localhost:4000/api`)
- **Auth token:** Stored in `localStorage` under key `astalakshimi.auth_token`
- **Request method:** Automatically attaches `Authorization: Bearer <token>` header
- **Error handling:** Parses JSON error responses from backend, extracts `message` and `errors` fields
- **Namespaced methods:** `apiClient.auth.*` , `apiClient.profiles.*` , `apiClient.media.*` , `apiClient.preferences.*`

5.4 State Management

Concern	Mechanism	Storage
Auth token	<code>localStorage</code> via <code>ApiClient</code>	<code>astalakshimi.auth_token</code>
Profile data (signup wizard)	<code>sessionStorage</code> via <code>profile-store.ts</code>	<code>astalakshimi.profile</code>
Server data caching	TanStack React Query	In-memory (30s stale time)
Shortlist, interests, notifications	<code>localStorage</code> (mock/client-side)	Various <code>astalakshimi.*</code> keys
Membership plan	<code>localStorage</code> (mock)	<code>astalakshimi.plan</code>
Settings	<code>localStorage</code> (mock)	<code>astalakshimi.settings</code>

[!WARNING] Most activity features (shortlists, interests, notifications, settings, plans) currently use **client-side `localStorage`-based mock data** from `user-activity.ts` and `plans.ts`. These do not call backend APIs. Match data comes from hardcoded arrays in `matches.ts`.

5.5 React Query Hooks (`queries.ts`)

Hook	Data Source	Backend API?
<code>useProfileQuery</code>	<code>sessionStorage</code> → <code>loadProfile()</code>	✗ Local
<code>useSaveProfileMutation</code>	<code>sessionStorage</code> + <code>apiClient.profiles.completeRegistration()</code>	✓ Yes
<code>useMatchesQuery</code>	Hardcoded <code>MATCHES</code> array	✗ Mock
<code>usePaidQuery</code>	<code>localStorage</code>	✗ Mock
<code>useSubscriptionQuery</code>	<code>localStorage</code>	✗ Mock
<code>useInvoicesQuery</code>	<code>localStorage</code>	✗ Mock
<code>useNotificationsQuery</code>	<code>localStorage</code>	✗ Mock
<code>useShortlistQuery</code>	<code>localStorage</code>	✗ Mock
<code>useInterestsQuery</code>	<code>localStorage</code>	✗ Mock
<code>useSettingsQuery</code>	<code>localStorage</code>	✗ Mock

6. Backend Architecture

6.1 Technology Stack

Concern	Technology	Version
Framework	NestJS	11.0.1
HTTP Platform	Express (via <code>@nestjs/platform-express</code>)	—
Authentication	Passport + passport-jwt + <code>@nestjs/jwt</code>	—
ORM	Drizzle ORM	0.38.4
Database Driver	postgres.js	3.4.5
Validation	Zod (via <code>@astalakshimi/validation</code>)	3.24.2
Cloud Storage	AWS SDK v3 (<code>@aws-sdk/client-s3</code> , <code>@aws-sdk/s3-request-presigner</code>)	3.726.0
Configuration	<code>@nestjs/config</code> with <code>registerAs</code> namespaced configs	4.0.0

6.2 Module Architecture

The NestJS application is organized into feature modules. Only modules listed in `app.module.ts` are actually loaded:

Active Modules (imported in AppModule):

Module	Status	Files
DatabaseModule	✔ Implemented	database.module.ts , database.constants.ts
HealthModule	✔ Implemented	health.controller.ts
AuthModule	✔ Implemented	auth.controller.ts , auth.service.ts , jwt.strategy.ts
ProfilesModule	✔ Implemented	profiles.controller.ts , profiles.service.ts
PreferencesModule	✔ Implemented	preferences.controller.ts , preferences.service.ts
MediaModule	✔ Implemented	media.controller.ts , media.service.ts , s3.provider.ts

Scaffolded Modules (files exist but are 0 bytes, NOT imported in AppModule):

Module	Purpose
SearchModule	Profile search with filters
InterestsModule	Send/receive/accept/decline interest requests
ShortlistsModule	Save/unsave profiles
MessagingModule	In-app chat after mutual interest
NotificationsModule	In-app, SMS, email notification channels
PaymentsModule	Razorpay/PhonePe payment processing
SubscriptionsModule	Plan management and entitlement checking
AdminModule	Admin panel for moderation
ModerationModule	Content and profile moderation
UsersModule	User management
HoroscopeModule	Horoscope-specific operations
JobsModule	Background job processing

6.3 API Endpoints (Implemented)

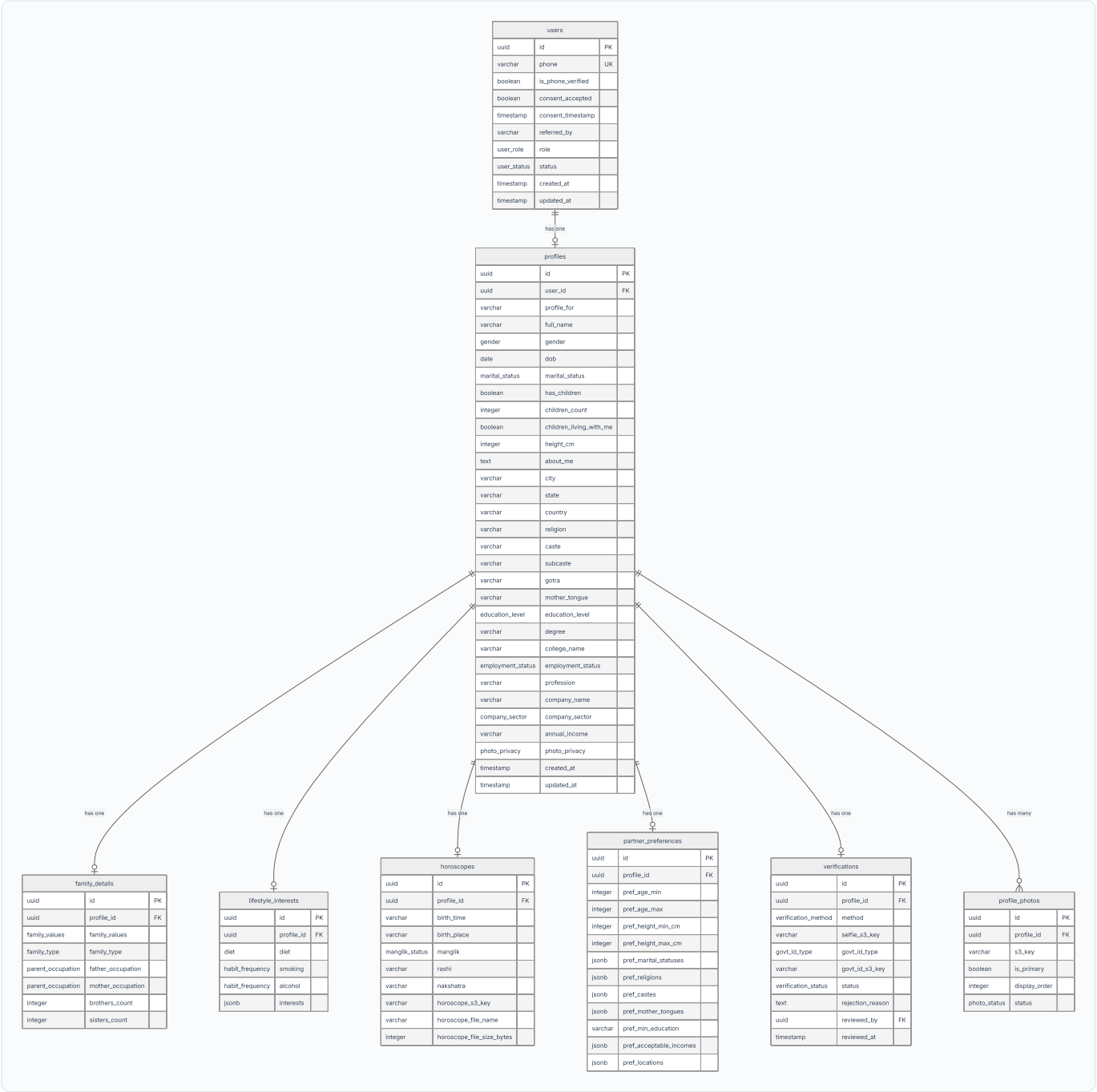
Method	Endpoint	Auth	Controller	Handler
POST	/api/auth/send-otp	✗ Public	AuthController	sendOtp()
POST	/api/auth/verify-otp	✗ Public	AuthController	verifyOtp()
GET	/api/auth/me	✔ JWT	AuthController	getMe()
POST	/api/profiles/complete-registration	✔ JWT	ProfilesController	completeRegistration()
GET	/api/profiles/me	✔ JWT	ProfilesController	getMyProfile()
GET	/api/profiles/:id	✗ Public	ProfilesController	getProfileById()
GET	/api/preferences/me	✔ JWT	PreferencesController	getMyPreferences()
PUT	/api/preferences/me	✔ JWT	PreferencesController	updateMyPreferences()
POST	/api/media/upload-url	✔ JWT	MediaController	getUploadUrl()
POST	/api/media/confirm-photo	✔ JWT	MediaController	confirmPhoto()
POST	/api/media/confirm-verification	✔ JWT	MediaController	confirmVerification()
POST	/api/media/confirm-horoscope	✔ JWT	MediaController	confirmHoroscope()
DELETE	/api/media/photos/:id	✔ JWT	MediaController	deletePhoto()
GET	/api/health	✗ Public	HealthController	check()

7. Database Architecture

7.1 Technology

Concern	Implementation
Database Engine	PostgreSQL (local development via <code>postgresql://...@localhost:5432/astalakshimi</code>)
ORM	Drizzle ORM 0.38.4
Driver	postgres.js 3.4.5
Connection Pooling	Built-in postgres.js pool: <code>max: 10</code> , <code>idle_timeout: 20s</code> , <code>connect_timeout: 10s</code>
SSL	Conditional — enabled for RDS or <code>sslmode=require</code> URLs
Migrations	Drizzle Kit (<code>drizzle-kit generate</code> / <code>drizzle-kit push</code>)
Schema Location	<code>packages/database/src/schema/</code>
Config	<code>drizzle.config.ts</code>

7.2 Entity-Relationship Diagram



7.3 Database Indexes

Table	Index Name	Columns	Purpose
profiles	profiles_search_idx	gender, religion, caste, city	Composite search filtering
profiles	profiles_dob_idx	dob	Age-based filtering
profile_photos	profile_photos_profile_idx	profile_id, display_order	Photo ordering per profile

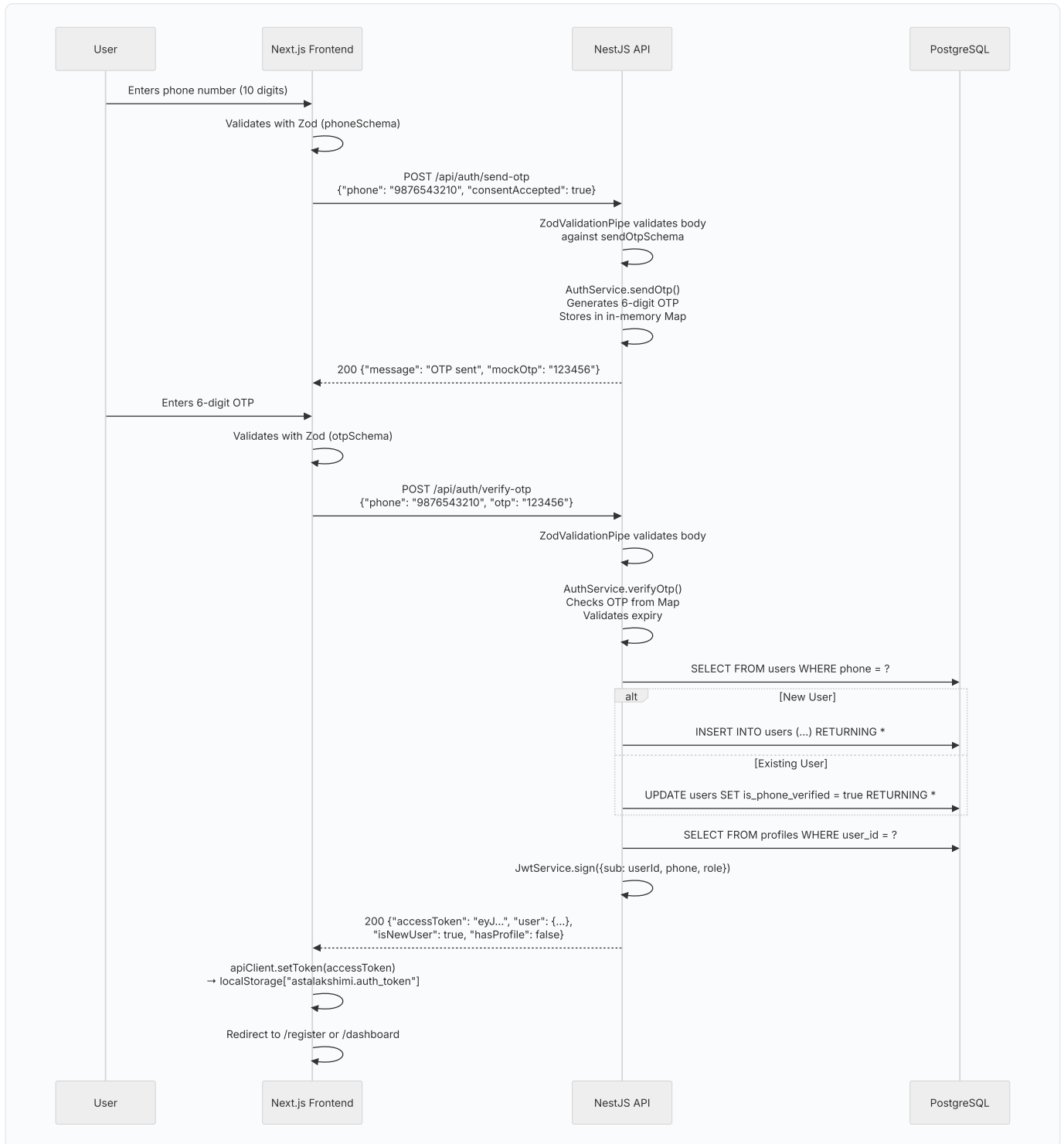
7.4 PostgreSQL Enums (Custom Types)

Enum	Values
user_role	member , admin , moderator
user_status	active , suspended , deactivated
gender	Male , Female , Other
marital_status	Never Married , Divorced , Widowed , Awaiting Divorce
education_level	Bachelors , Masters , Doctorate , Diploma , High School
employment_status	Employed , Business Owner , Freelancer , Not Working
company_sector	Private , Govt , MNC , Startup , Business
photo_privacy	blurred , accepted , visible
family_values	Traditional , Moderate , Liberal
family_type	Nuclear , Joint , Extended
parent_occupation	Employed , Business , Retired , Homemaker , Passed Away
diet	Vegetarian , Non-vegetarian , Eggetarian , Jain , Vegan
habit_frequency	Never , Occasionally , Regularly , Planning to quit
manglik_status	Yes , No , Don't Know , Both
verification_method	selfie , govt_id
verification_status	idle , pending , verified , rejected
govt_id_type	Aadhaar , PAN card , Passport , Driving licence , Voter ID
photo_status	pending , approved , rejected

8. Authentication & Authorization

8.1 Authentication Flow

The system uses **phone number + OTP** authentication with **JWT bearer tokens**.



8.2 JWT Token Details

Property	Value
Algorithm	HS256 (default)
Secret	<code>JWT_SECRET</code> env var
Expiry	<code>JWT_EXPIRES_IN</code> (default <code>30d</code>)
Payload	<code>{ sub: userId, phone: string, role: "member" "admin" "moderator" }</code>
Extraction	<code>Authorization: Bearer <token></code> header
Storage (frontend)	<code>localStorage</code> key <code>astalakshimi.auth_token</code>

8.3 OTP Storage

[!WARNING] OTPs are stored in an **in-memory** `Map<string, PendingOtp>` inside `AuthService`. This means:

- OTPs are lost on server restart
- OTPs cannot be shared across multiple API instances
- Recommended:** Migrate to Redis or a database-backed OTP store for production

Property	Value
TTL	<code>OTP_TTL_SECONDS</code> (default 300 = 5 minutes)
Mock Mode	<code>MOCK_OTP_ENABLED=true</code> → always returns <code>DEFAULT MOCK_OTP</code> (123456)
Production	Would generate <code>Math.floor(100000 + Math.random() * 900000)</code>
SMS Delivery	Not implemented — SMS provider (e.g., Twilio, MSG91) is not integrated

8.4 Request Authentication (JWT Strategy)

For every protected route, the `JwtAuthGuard` triggers `JwtStrategy.validate()`:

- Extract Bearer token from `Authorization` header
- Verify JWT signature and expiry
- Decode `sub` (userid) from payload
- Query `users` table: `SELECT id, phone, role, status FROM users WHERE id = sub`
- Check `user.status === 'active'` — throw `UnauthorizedException` if not
- Return `UserSession { userId, phone, role }` → attached to `request.user`

The `@CurrentUser()` decorator extracts `request.user` for use in controllers.

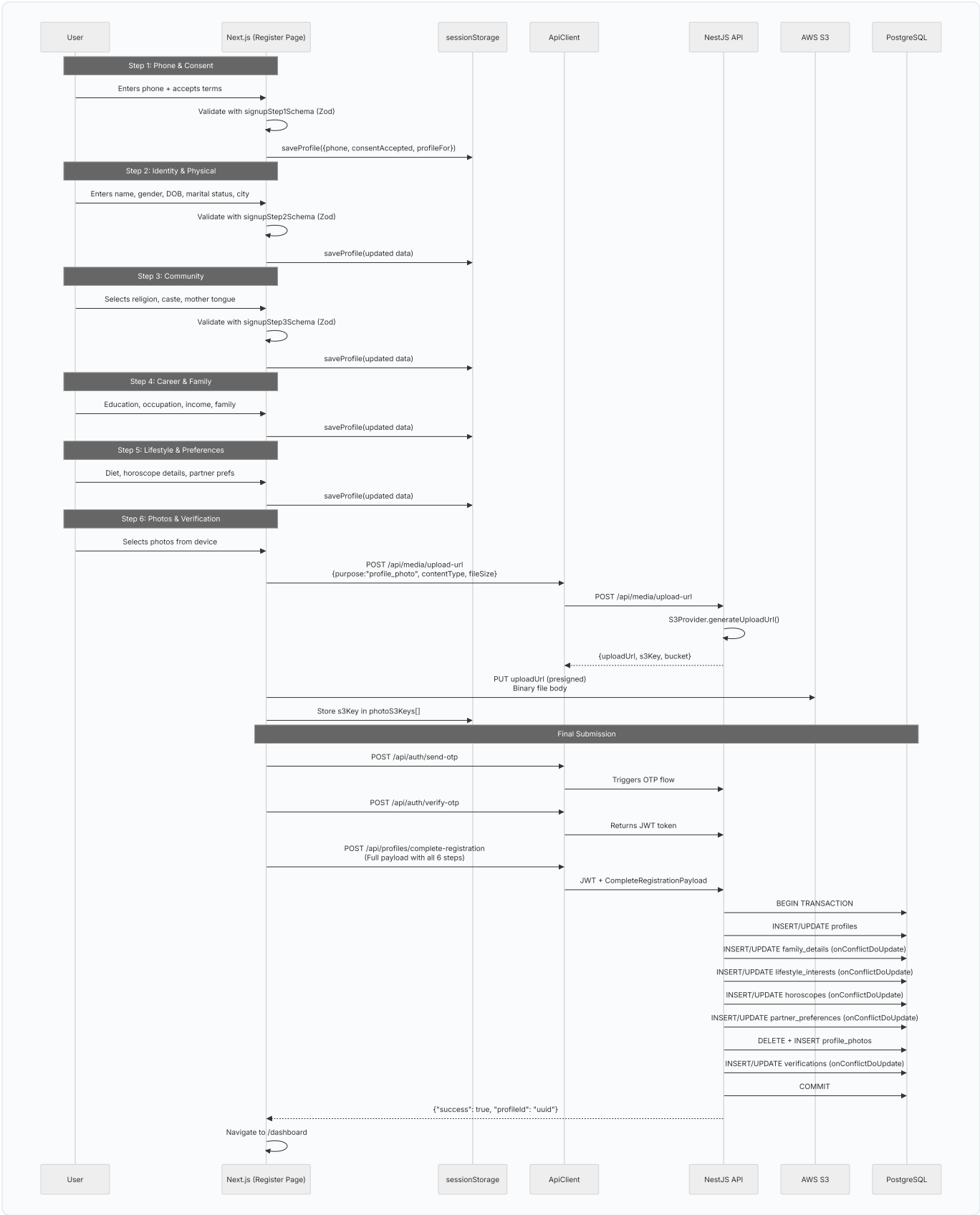
8.5 Authorization

Mechanism	Status
Role-based guards	Scaffolded — <code>roles.guard.ts</code> and <code>@Roles().decorator</code> exist but are not yet implemented (0-byte)
Entitlement guards	Scaffolded — <code>entitlement.guard.ts</code> exists but is not yet implemented (0-byte)
Resource ownership	Implemented inline — controllers use <code>user.userId</code> from JWT to scope queries

9. End-to-End Data Flow

9.1 Flow 1: User Registration (OTP + Multi-Step Profile)

This is the most complex flow in the system, spanning 6 steps in the frontend wizard and a single atomic backend transaction.



Data at Each Stage:

Frontend → API Request:

```

POST /api/profiles/complete-registration
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type: application/json

{
  "phone": "9876543210",
  "profileFor": "Myself",
  "fullName": "Priya Sharma",
  "gender": "Female",
  "dobDay": "15",
  "dobMonth": "06",
  "dobYear": "1998",
  "maritalStatus": "Never Married",
  "heightCm": 162,
  "city": "Chennai",
  "state": "Tamil Nadu",
  "country": "India",
  "religion": "Hindu",
  "caste": "Brahmin",
  "motherTongue": "Tamil",
  "educationLevel": "Masters",
  "degree": "M.Tech Computer Science",
  "employmentStatus": "Employed",
  "profession": "Software Engineer",
  "annualIncome": "₹15 - 20 Lakh",
  "familyValues": "Moderate",
  "familyType": "Nuclear",
  "fatherOccupation": "Retired",
  "motherOccupation": "Homemaker",
  "brothersCount": 1,
  "sistersCount": 0,
  "diet": "Vegetarian",
  "manglik": "Don't Know",
  "prefAgeMin": 26,
  "prefAgeMax": 32,
  "prefReligions": ["Hindu"],
  "photoS3Keys": ["profiles/uuid/photos/abc.webp"],
  "photoPrivacy": "blurred",
  "verificationMethod": "selfie",
  "selfieS3Key": "verifications/uuid/selfie-xyz.jpg"
}

```

Database Operations (inside transaction):

```

-- 1. Insert profile
INSERT INTO profiles (user_id, profile_for, full_name, gender, dob, ...)
VALUES ($1, 'Myself', 'Priya Sharma', 'Female', '1998-06-15', ...)
RETURNING *;

-- 2. Upsert family details
INSERT INTO family_details (profile_id, family_values, ...)
VALUES ($1, 'Moderate', ...)
ON CONFLICT (profile_id) DO UPDATE SET family_values = 'Moderate', ...;

-- 3. Upsert lifestyle
INSERT INTO lifestyle_interests (profile_id, diet, ...)
VALUES ($1, 'Vegetarian', ...)
ON CONFLICT (profile_id) DO UPDATE SET diet = 'Vegetarian', ...;

-- 4. Upsert horoscope
INSERT INTO horoscopes (profile_id, manglik, ...)
VALUES ($1, 'Don't Know', ...)
ON CONFLICT (profile_id) DO UPDATE SET manglik = 'Don't Know', ...;

-- 5. Upsert partner preferences
INSERT INTO partner_preferences (profile_id, pref_age_min, pref_age_max, ...)
VALUES ($1, 26, 32, ...)
ON CONFLICT (profile_id) DO UPDATE SET pref_age_min = 26, ...;

-- 6. Replace photos
DELETE FROM profile_photos WHERE profile_id = $1;
INSERT INTO profile_photos (profile_id, s3_key, is_primary, display_order, status)
VALUES ($1, 'profiles/.../abc.webp', true, 0, 'pending');

-- 7. Upsert verification
INSERT INTO verifications (profile_id, method, selfie_s3_key, status)
VALUES ($1, 'selfie', 'verifications/.../selfie-xyz.jpg', 'pending')
ON CONFLICT (profile_id) DO UPDATE SET method = 'selfie', ...;

```

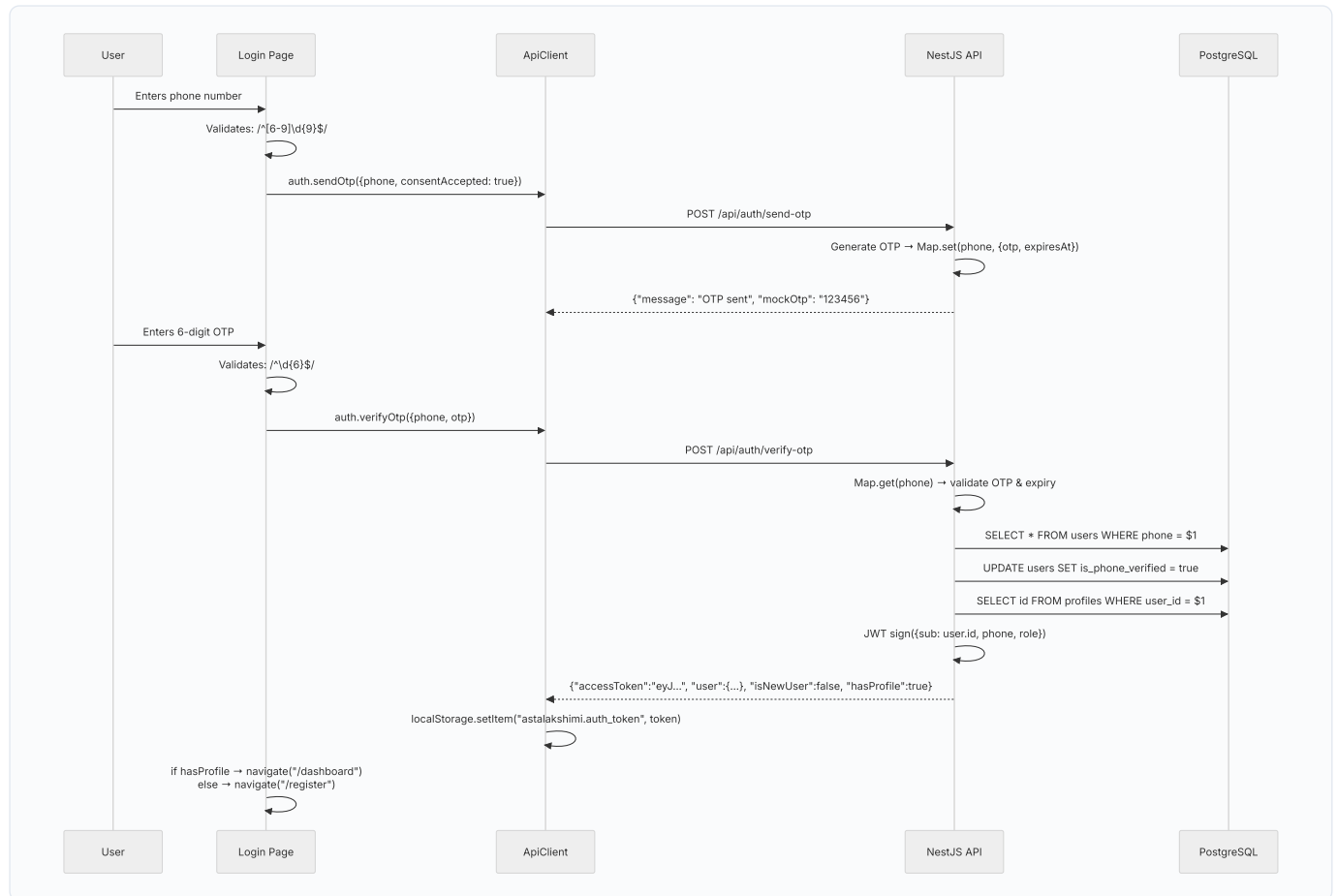
API Response:

```

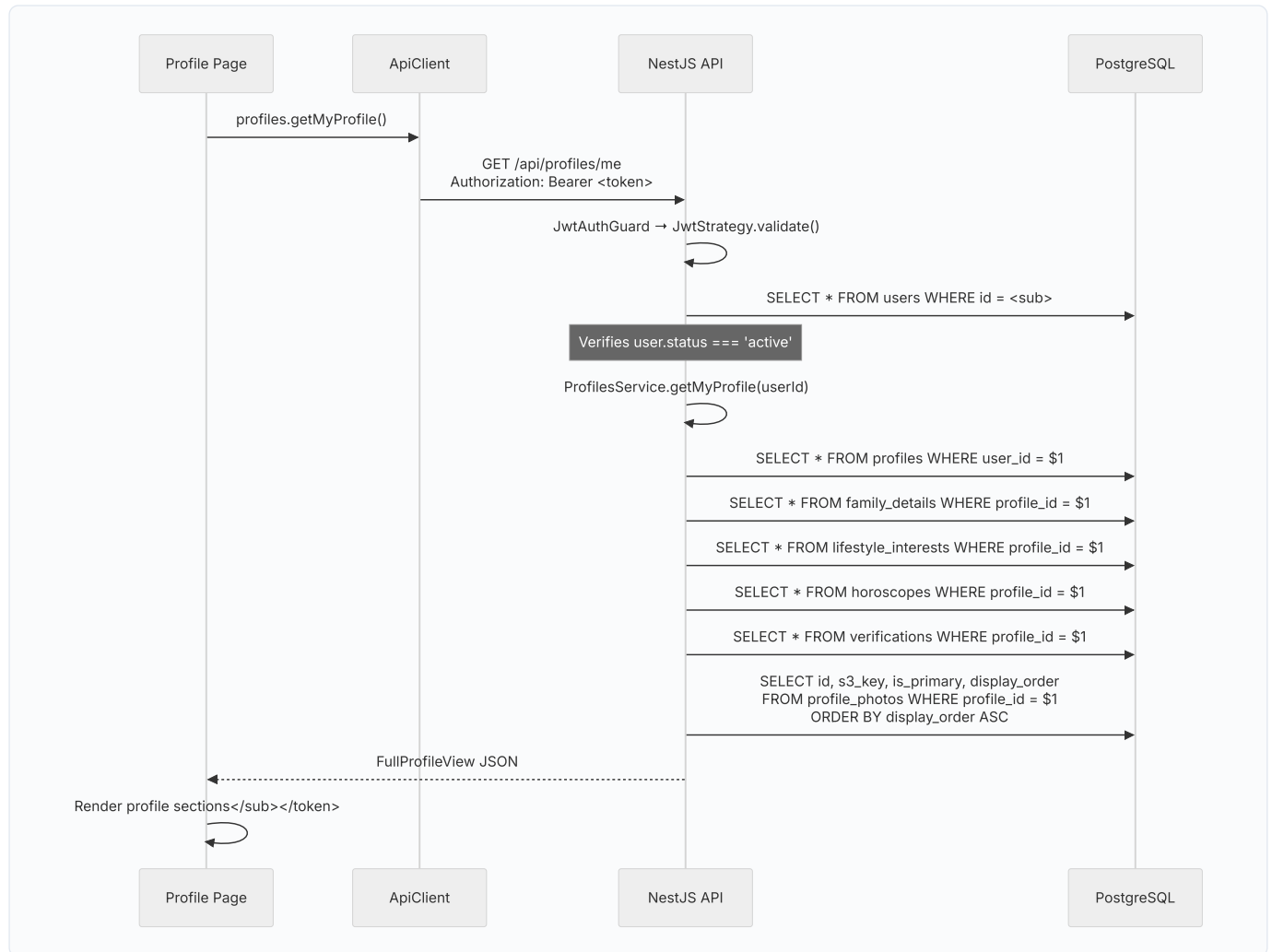
{
  "success": true,
  "message": "Profile registration completed successfully",
  "profileId": "a1b2c3d4-e5f6-..."
}

```

9.2 Flow 2: User Login



9.3 Flow 3: Get My Profile



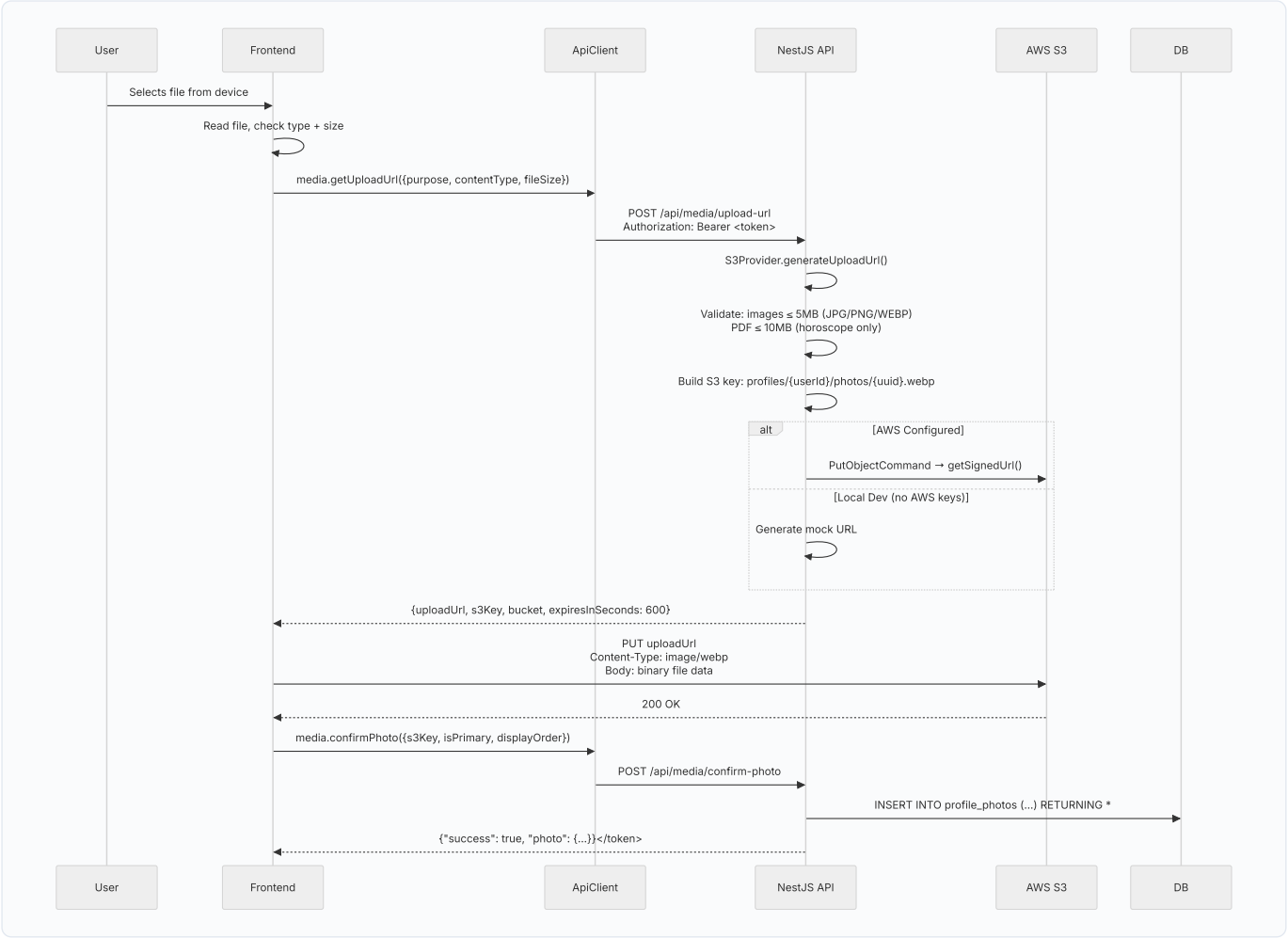
Response shape (FullProfileView):

```

{
  "profile": { "id": "...", "fullName": "Priya Sharma", "gender": "Female", ... },
  "family": { "familyValues": "Moderate", "familyType": "Nuclear", ... },
  "lifestyle": { "diet": "Vegetarian", "interests": ["📖 Reading", "✈️ Travel"] },
  "horoscope": { "manglik": "Don't Know", "rashi": "Meena", ... },
  "photos": [{ "id": "...", "s3Key": "profiles/.../photo.webp", "isPrimary": true, "displayOrder": 0 }],
  "verificationStatus": "pending"
}

```

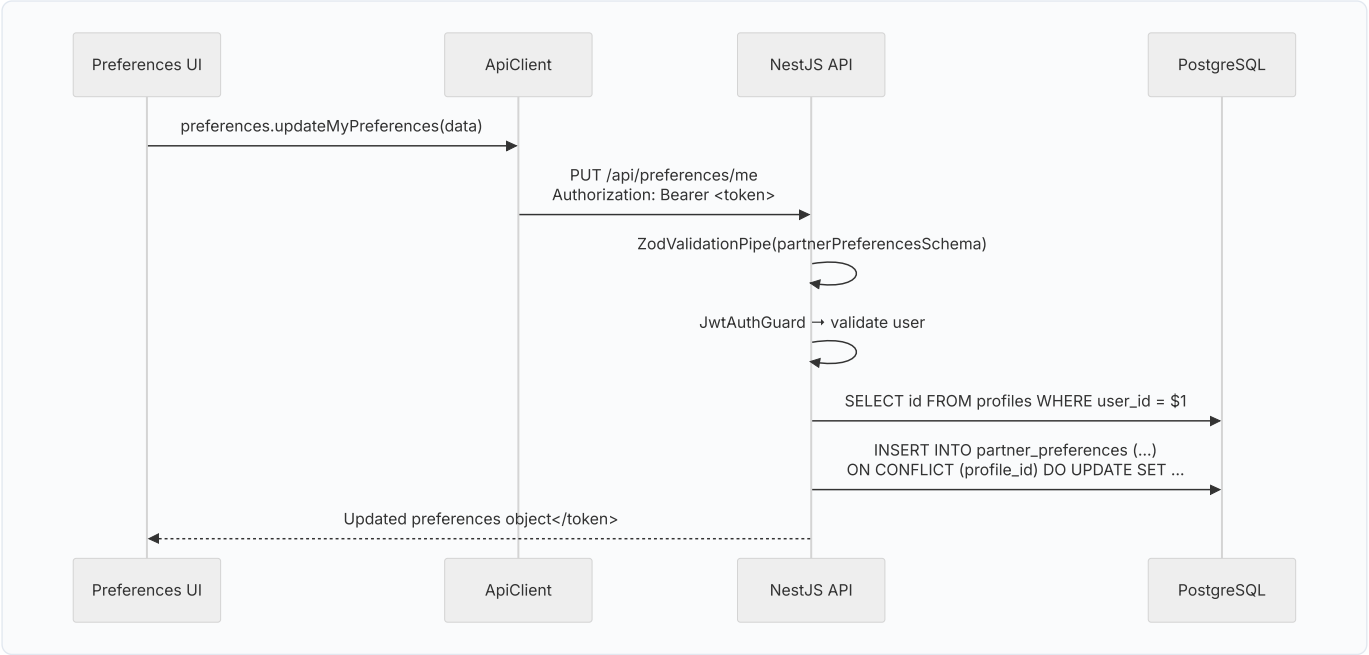
9.4 Flow 4: Media Upload (Photos, Verification, Horoscope)



S3 Key Patterns:

Purpose	Bucket	Key Pattern
profile_photo	astalakshimi-media-dev	profiles/{userId}/photos/{uuid}.{ext}
horoscope	astalakshimi-media-dev	profiles/{userId}/horoscopes/{uuid}.pdf
selfie	astalakshimi-vault-dev	verifications/{userId}/selfie-{uuid}.{ext}
govt_id	astalakshimi-vault-dev	verifications/{userId}/govt-id-{uuid}.{ext}

9.5 Flow 5: Update Partner Preferences



9.6 Flow 6: Health Check

```
GET /api/health

Response:
{
  "status": "ok",
  "database": "healthy", // or "disconnected"
  "uptime": 12345.678,
  "timestamp": "2026-08-19T16:06:00.000Z"
}
```

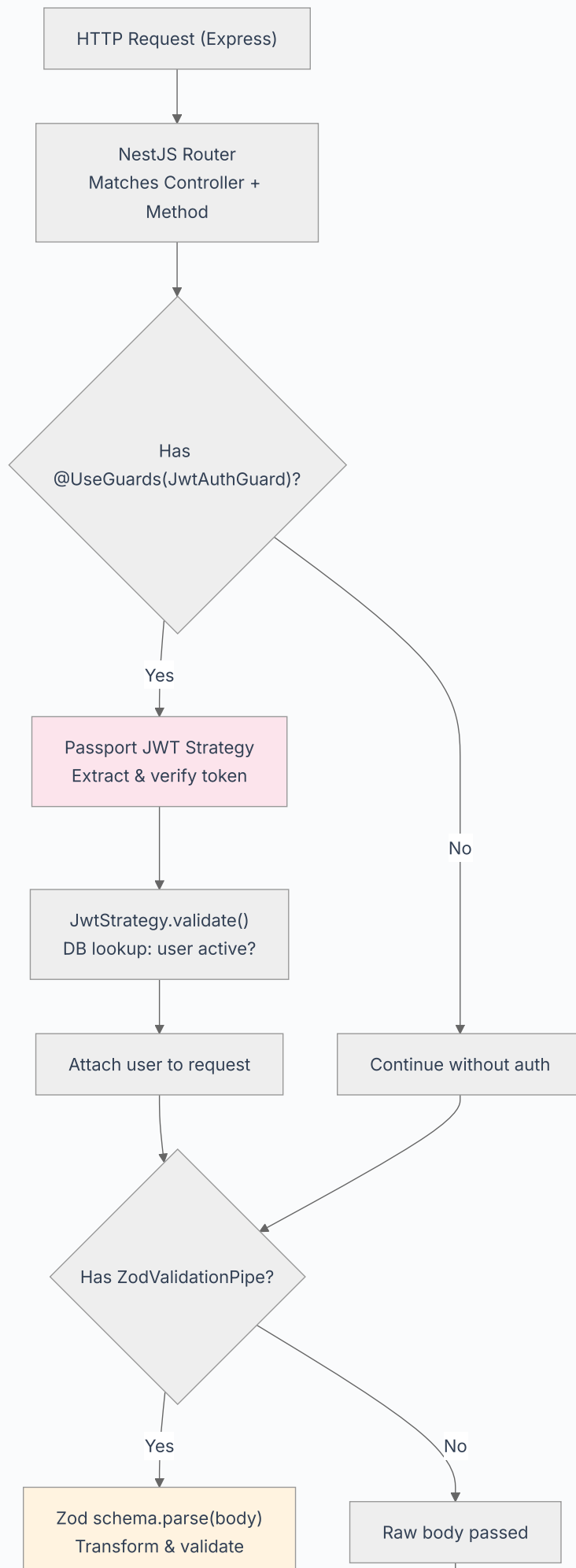
The health endpoint executes `SELECT 1` to verify database connectivity.

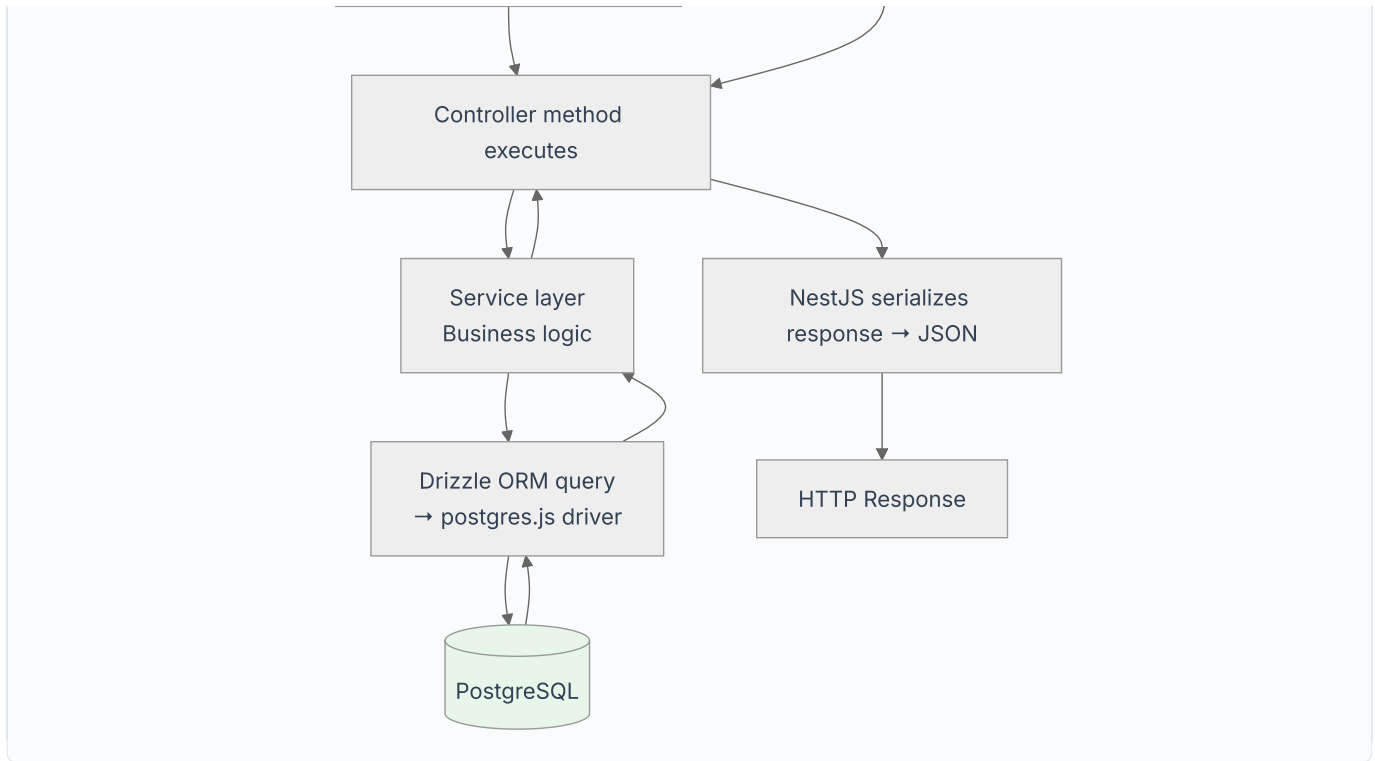
10. Frontend → Backend API Mapping

User Action	Frontend Component	HTTP Method	API Endpoint	Request Data	Backend Handler	Data
Enter phone on login	Login page	POST	/api/auth/send-otp	{phone, consentAccepted}	AuthController.sendOtp() → AuthService.sendOtp()	None
Submit OTP	Login page	POST	/api/auth/verify-otp	{phone, otp}	AuthController.verifyOtp() → AuthService.verifyOtp()	SELECT user profile
Complete registration	Register wizard	POST	/api/profiles/complete-registration	CompleteRegistrationPayload (40+ fields)	ProfilesController.completeRegistration() → ProfilesService.completeRegistration()	TRAFFIC INCREASE
View my profile	Profile page	GET	/api/profiles/me	None	ProfilesController.getMyProfile() → ProfilesService.getMyProfile()	6 SECONDS across all devices
View another profile	Profile card click	GET	/api/profiles/:id	Path param: id	ProfilesController.getProfileById() → ProfilesService.getProfileById()	6 SECONDS
Get my preferences	Preferences page	GET	/api/preferences/me	None	PreferencesController.getMyPreferences()	SELECT user preferences
Update preferences	Preferences form	PUT	/api/preferences/me	PartnerPreferencesInput	PreferencesController.updateMyPreferences()	SELECT user preferences UPSIDE DOWN partner preferences
Upload file (photo/doc)	Photo picker	POST	/api/media/upload-url	{purpose, contentType, fileSize}	MediaController.getUploadUrl() → S3Provider.generateUploadUrl()	None
Upload to S3	After presigned URL	PUT	S3 presigned URL	Binary file	AWS S3	None
Confirm photo upload	After S3 upload	POST	/api/media/confirm-photo	{s3Key, isPrimary, displayOrder}	MediaController.confirmPhoto() → MediaService.confirmPhoto()	INCREASE profile photo
Submit verification	Verification step	POST	/api/media/confirm-verification	{method, selfieS3Key?, govtIdType?, govtIdS3Key?}	MediaController.confirmVerification()	UPSIDE DOWN verification
Upload horoscope	Horoscope step	POST	/api/media/confirm-horoscope	{horoscopeS3Key, fileName, fileSizeBytes}	MediaController.confirmHoroscope()	UPSIDE DOWN horoscope
Delete photo	Photo gallery	DELETE	/api/media/photos/:id	Path param: id	MediaController.deletePhoto()	DELETE profile photo delete
Check auth status	App load	GET	/api/auth/me	None	AuthController.getMe() → AuthService.getMe()	SELECT user preferences
Discover matches	Dashboard	—	No backend API	—	—	—
Send interest	Profile card	—	No backend API	—	—	—
Shortlist profile	Profile card	—	No backend API	—	—	—
View notifications	Notifications page	—	No backend API	—	—	—
Purchase plan	Plans page	—	No backend API	—	—	—
Update settings	Settings page	—	No backend API	—	—	—

11. Backend Request Lifecycle





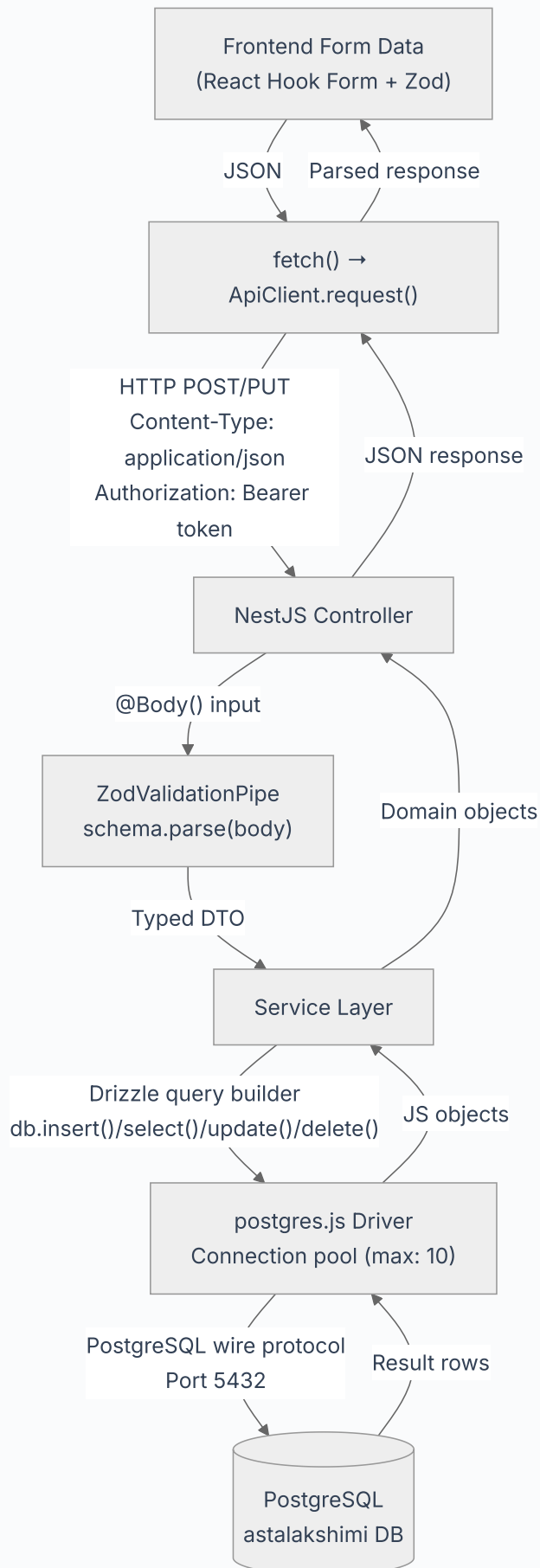


Layer Details

Layer	Responsibility	Key Files	Input	Output
Express/NestJS Router	Match URL to controller method	Auto-generated from <code>@Controller()</code> + <code>@Get/@Post/@Put/@Delete</code> decorators	HTTP Request	Matched handler
CORS Middleware	Allow cross-origin requests from frontend	<code>main.ts</code>	Request origin	Allow/block
Global Exception Filter	Catch all unhandled exceptions	<code>http-exception.filter.ts</code>	Any thrown exception	<code>{statusCode, message, errors?, timestamp, path}</code>
JwtAuthGuard	Enforce authentication	<code>auth.guard.ts</code>	Bearer token	<code>UserSession</code> on <code>req.user</code> or 401
JwtStrategy	Verify JWT + load user from DB	<code>jwt.strategy.ts</code>	JWT payload <code>{sub, phone, role}</code>	<code>UserSession {userId, phone, role}</code>
ZodValidationPipe	Validate request body against Zod schema	<code>zod-validation.pipe.ts</code>	Raw body	Parsed & validated body or 400 with field errors
Controller	Route handling, delegate to service	Various <code>*.controller.ts</code>	Validated input + <code>UserSession</code>	Service result
Service	Business logic, data transformation	Various <code>*.service.ts</code>	Domain input	Domain output
Drizzle ORM	Type-safe SQL query building	Schema in <code>packages/database/src/schema/</code>	Query builder calls	Query results
postgres.js	PostgreSQL wire protocol driver	Via <code>createDbClient()</code> in <code>client.ts</code>	SQL queries	Raw results

12. Database Data Flow



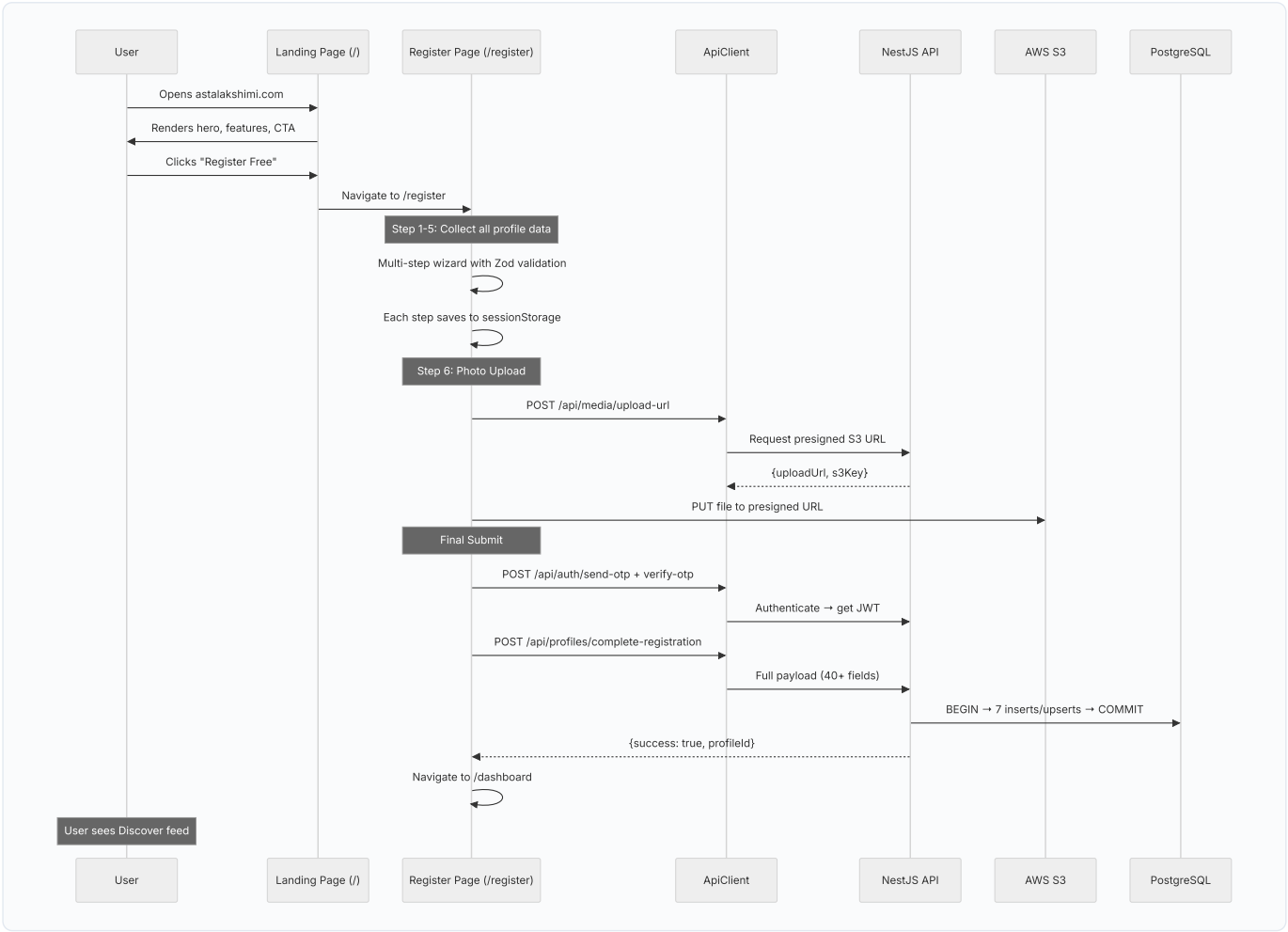


Connection Details:

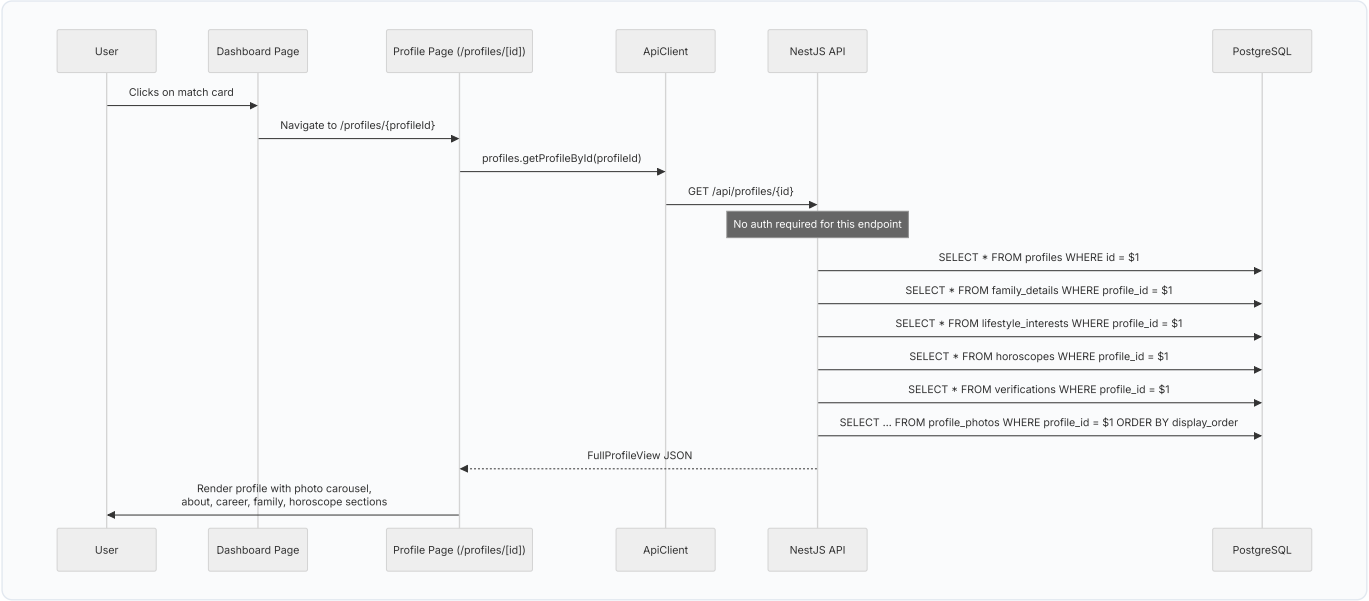
Property	Value
Connection String	<code>DATABASE_URL</code> env var (e.g., <code>postgresql://user@localhost:5432/astalakshimi</code>)
Driver	<code>postgres</code> (postgres.js v3.4.5)
Pool Size	<code>max: 10</code> connections
Idle Timeout	<code>20</code> seconds
Connect Timeout	<code>10</code> seconds
SSL	Auto-detected for RDS domains or <code>sslmode=require</code>
ORM	Drizzle ORM with full schema introspection
Credentials Location	Root <code>.env</code> file → loaded via <code>@nestjs/config</code> → <code>ConfigService.get('database.url')</code>

13. Major User Flow Sequence Diagrams

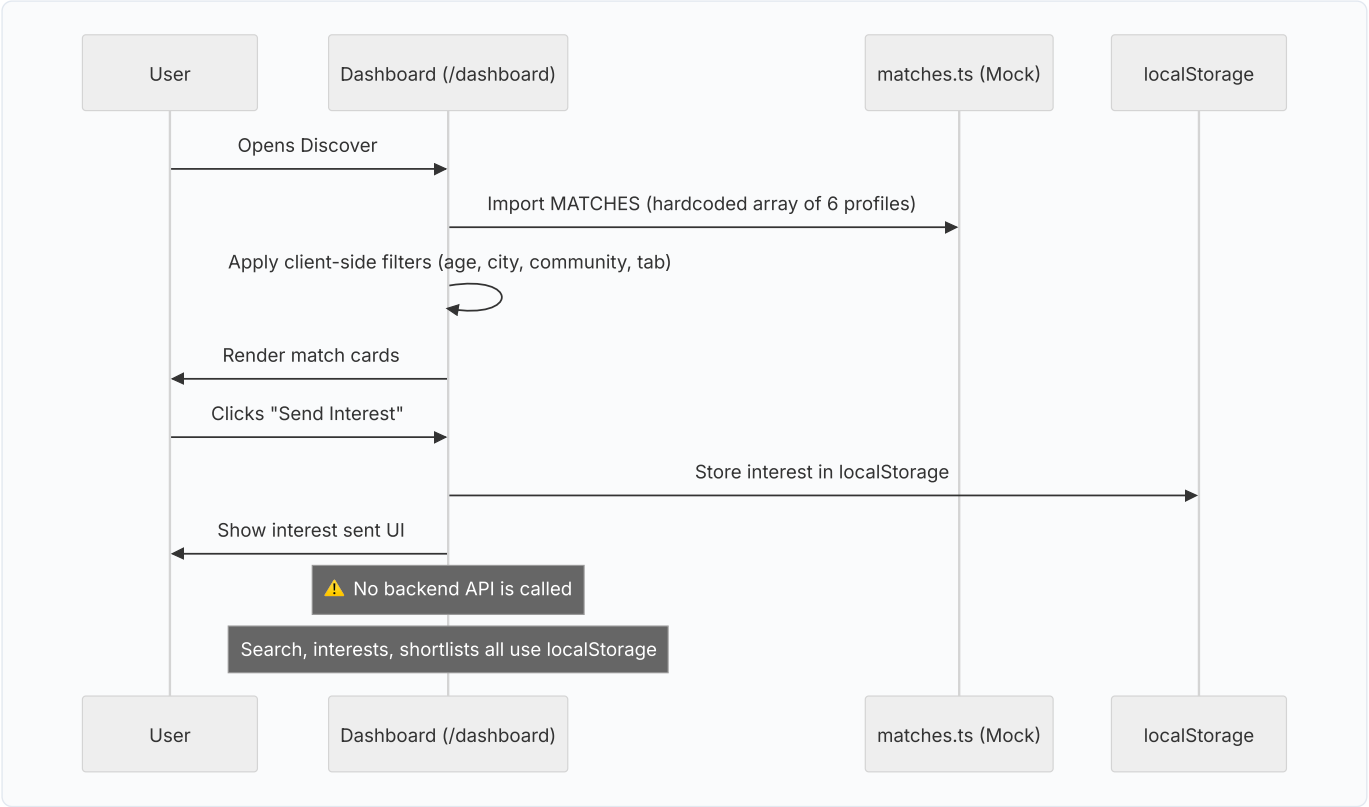
13.1 Landing Page → Registration → Dashboard



13.2 Profile View Flow



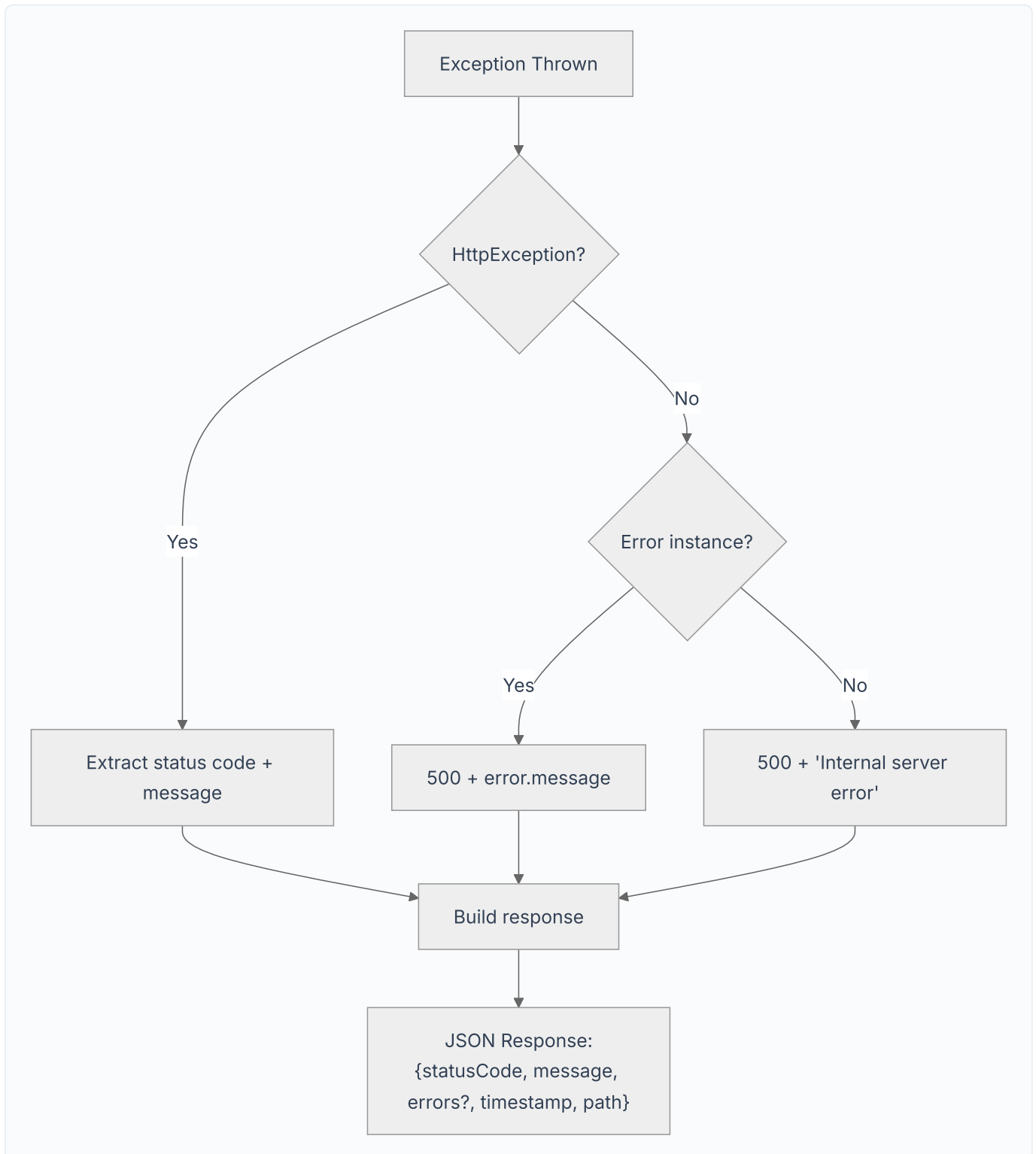
13.3 Discover / Search Flow (Currently Mock)



14. Error Handling Flow

14.1 Backend Error Pipeline

The `GlobalExceptionHandler` catches all exceptions:



14.2 Error Scenarios

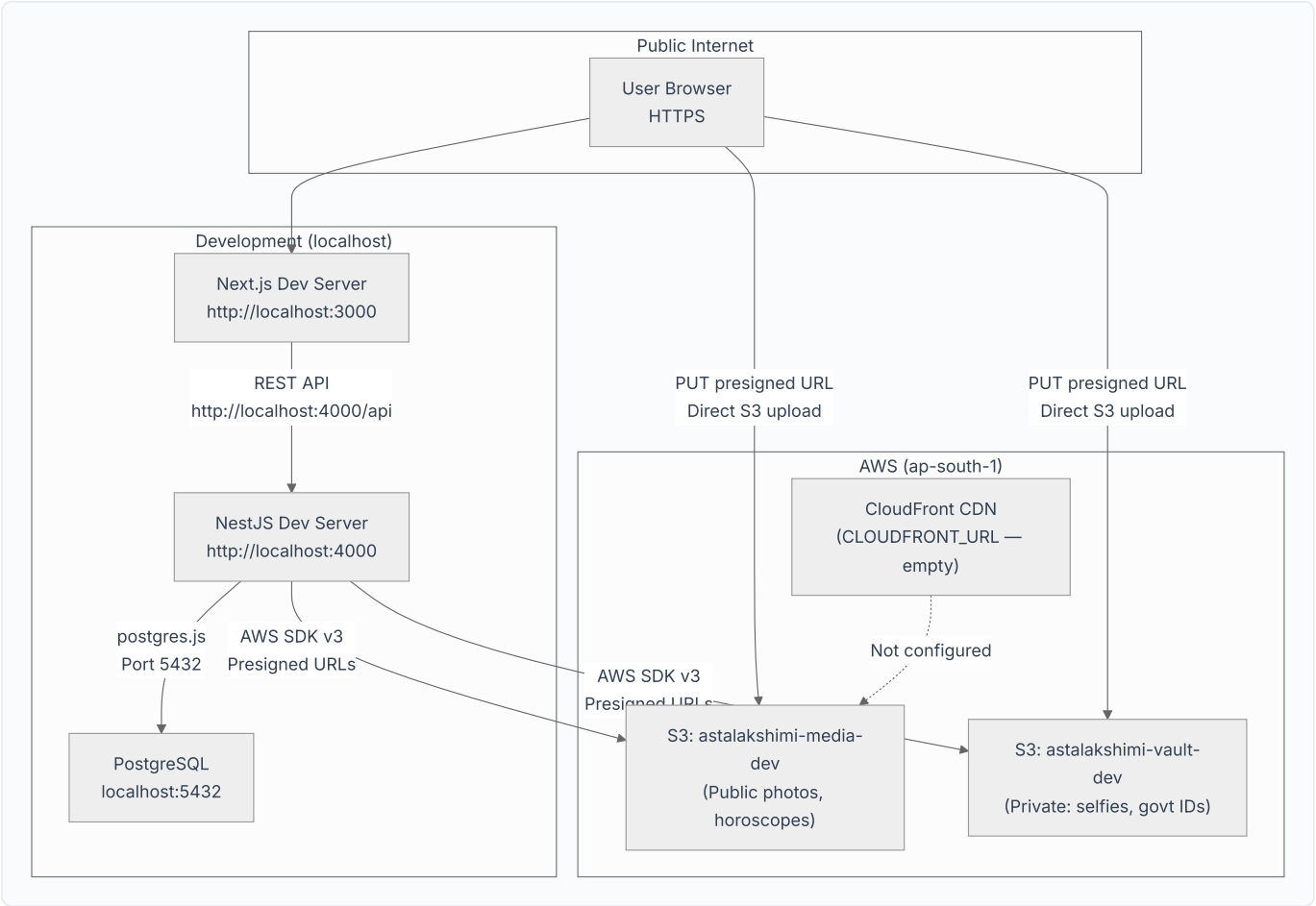
Scenario	Backend Behavior	HTTP Status	Error Response	Frontend Behavior
Frontend validation fails	Not reached	—	—	Zod schema shows inline field errors via React Hook Form
Invalid OTP	<code>BadRequestException</code>	400	<code>{message: "Invalid OTP. Please check and try again."}</code>	<code>ApiClient</code> throws Error, caught by mutation's <code>onError</code> , displayed as toast/alert
Expired OTP	<code>BadRequestException</code>	400	<code>{message: "OTP has expired. Please request a new OTP."}</code>	Error shown, user re-requests OTP
No pending OTP	<code>BadRequestException</code>	400	<code>{message: "No pending OTP request found..."}</code>	Error shown
Missing/invalid JWT	<code>UnauthorizedException</code>	401	<code>{message: "Authentication required. Please log in."}</code>	<code>ApiClient</code> throws, could redirect to <code>/login</code>
User suspended/deactivated	<code>UnauthorizedException</code>	401	<code>{message: "User account is not active or no longer exists"}</code>	Redirect to login
Profile not found	<code>NotFoundException</code>	404	<code>{message: "Profile not found"}</code>	Show error UI
Zod validation fails	<code>BadRequestException</code>	400	<code>{message: "Validation failed", errors: [{field: "phone", message: "..."}]}</code>	Field-level errors displayed
Invalid file type/size	<code>BadRequestException</code>	400	<code>{message: "Photos must be JPG, PNG, or WEBP under 5 MB."}</code>	Error shown
S3 presigned URL failure	<code>BadRequestException</code>	400	<code>{message: "Could not generate secure upload URL..."}</code>	Retry prompt
Database unavailable	Unhandled → caught by filter	500	<code>{statusCode: 500, message: "Internal server error"}</code>	Generic error
Unknown server error	Caught by filter	500	<code>{statusCode: 500, message: error.message}</code>	Generic error

14.3 Frontend Error Handling

In `api-client.ts`:

```
if (!response.ok) {
  // Attempt to parse JSON error body
  const errorData = await response.json();
  errorMessage = errorData.message || 'An unexpected error occurred';
  // Append field-level validation errors if present
  if (errorData.errors && Array.isArray(errorData.errors)) {
    const fieldErrors = errorData.errors.map(e => `${e.field}: ${e.message}`).join(', ');
    errorMessage = `${errorMessage} (${fieldErrors})`;
  }
  throw new Error(errorMessage);
}
```

15. Network Architecture



CORS Configuration ([main.ts](#))

Setting	Value
Origins	<code>CORS_ORIGIN</code> env var (default: <code>http://localhost:3000</code>)
Credentials	<code>true</code>
Methods	<code>GET, POST, PUT, PATCH, DELETE, OPTIONS</code>
Allowed Headers	<code>Content-Type, Authorization</code>

16. Deployment Architecture

16.1 Current State

Component	Configuration	Status
Frontend (Next.js)	<code>vercel.json</code> — framework: nextjs, installCommand: <code>npm ci</code>	✅ Configured for Vercel
Backend (NestJS)	<code>nest start --watch</code> (dev), <code>node dist/main</code> (prod)	⚠️ No deployment config
Database	Local PostgreSQL	⚠️ No cloud DB configured
Docker	<code>infrastructure/docker/</code> — empty <code>.gitkeep</code>	❌ Not implemented
Docker Compose	<code>infrastructure/docker-compose/</code> — empty <code>.gitkeep</code>	❌ Not implemented
CI/CD	None found	❌ Not implemented

16.2 Deployment Flow (Current)



16.3 Recommended Deployment Architecture

[!NOTE] **Proposed / Recommended** — Not currently implemented



17. Security Architecture

17.1 Security Boundaries

Component	Access	Risk Level
Landing page (/)	Public	Low
Login/Register pages	Public	Medium — OTP brute-force
Auth endpoints (/api/auth/*)	Public	High — OTP validation
Profile endpoints (/api/profiles/*)	Mixed (:id is public, me is JWT)	Medium
Media endpoints (/api/media/*)	JWT required	Medium
Preferences endpoints (/api/preferences/*)	JWT required	Low
S3 Media bucket	Presigned URL (time-limited)	Medium
S3 Vault bucket	Presigned URL (time-limited)	High — sensitive docs
PostgreSQL	Private (localhost / VPC)	High

17.2 Implemented Security

Measure	Status	Details
HTTPS/TLS	⚠️ Dev only (HTTP)	Vercel handles HTTPS in production for frontend
JWT Authentication	✅ Implemented	HS256, 30-day expiry, Bearer tokens
CORS	✅ Implemented	Restricted to configured origins
Input Validation	✅ Implemented	Zod schemas on frontend + backend
Presigned S3 URLs	✅ Implemented	10-minute expiry, scoped to specific S3 keys
S3 Bucket Separation	✅ Implemented	Public media vs private vault buckets
UUID Primary Keys	✅ Implemented	Non-sequential, non-guessable IDs
Resource Ownership	✅ Partially	Controllers use <code>userId</code> from JWT to scope profile queries
Password Storage	N/A	No passwords — OTP-only auth

17.3 Security Gaps

Risk	Severity	Details
No rate limiting	🔴 High	OTP endpoint has no brute-force protection. An attacker could try all 1M combinations
In-memory OTP storage	🔴 High	OTPs lost on restart, doesn't scale horizontally
JWT secret in .env	🟡 Medium	<code>JWT_SECRET</code> is a plaintext string — should use secrets manager in production
No CSRF protection	🟡 Medium	JWT in localStorage mitigates some risk but XSS could steal tokens
Mock OTP in production risk	🟡 Medium	<code>MOCK_OTP_ENABLED</code> defaults to <code>true</code> — must be explicitly disabled
No request signing	🟡 Medium	API requests are not signed beyond JWT
Profile <code>:id</code> endpoint is public	🟡 Medium	Anyone can fetch any profile by UUID without authentication
No SQL injection risk	✅ Safe	Drizzle ORM uses parameterized queries
No XSS in API responses	✅ Safe	JSON responses, no HTML rendering

18. Scalability & Reliability

18.1 Current Architecture Analysis

Concern	Current State	Bottleneck?
Frontend	Next.js on Vercel — auto-scales	✅ Good
Backend	Single NestJS process	🔴 Single point of failure
Database	Single PostgreSQL instance, 10-connection pool	🟡 Pool could exhaust under load
OTP Storage	In-memory Map	🔴 Not scalable, lost on restart
Session/Auth	Stateless JWT	✅ Good — no server-side session
File Uploads	Direct-to-S3 presigned URLs	✅ Good — offloads from API
Caching	None	🟡 All reads hit PostgreSQL
Background Jobs	Not implemented (scaffolded)	🟡 Missing for email, notifications
CDN	CloudFront env var exists but empty	🟡 Photos served from S3 directly

18.2 What Would Break First

- 1. **OTP in-memory Map** — Multiple API instances would have different OTP caches
- 2. **Database connections** — 10-connection pool shared by all concurrent requests; under heavy load, queries would queue
- 3. **Profile queries** — `getMyProfile()` and `getProfileById()` execute **6 sequential queries** per call (not batched/joined)
- 4. **No caching** — Repeated profile views always hit the database

18.3 Recommended Improvements

[!NOTE] **Proposed / Recommended** — Not currently implemented

Improvement	Priority	Details
Redis for OTP storage	🔴 Critical	Replace in-memory Map with Redis; supports TTL natively, scales horizontally
Rate limiting	🔴 Critical	Add <code>@nestjs/throttler</code> — especially on <code>/auth/send-otp</code> (e.g., 3 requests/minute/phone)
Database connection pool tuning	🟡 High	Increase <code>max</code> pool size for production; consider PgBouncer for connection pooling
Query optimization	🟡 High	Join profile-related tables in a single query instead of 6 sequential SELECTs
CloudFront CDN	🟡 High	Configure CloudFront for S3 media delivery to reduce latency
Horizontal scaling	🟡 Medium	Containerize API (Docker), deploy multiple instances behind load balancer
Read replicas	🟢 Low	Add PostgreSQL read replica for search/discover queries
Background job queue	🟡 Medium	Implement BullMQ or similar for notifications, verification reviews, email

19. Observability

19.1 Current State

Concern	Implemented?	Details
Application logs	⚠️ Minimal	NestJS <code>Logger</code> used in <code>AuthService</code> (<code>logger.log</code>), <code>S3Provider</code> (<code>logger.warn</code> / <code>logger.error</code>), and <code>Bootstrap</code>
Structured logging	❌ Not implemented	Logs are plain strings, not JSON-structured
Access logs	❌ Not implemented	No request logging middleware
Error tracking	⚠️ Basic	<code>GlobalHttpExceptionHandler</code> returns errors in HTTP responses but doesn't log to external service
Health check	✅ Implemented	<code>GET /api/health</code> — checks DB connectivity, returns uptime
Metrics	❌ Not implemented	No Prometheus, Datadog, or similar
Distributed tracing	❌ Not implemented	No OpenTelemetry or similar
Monitoring	❌ Not implemented	No uptime monitoring
Alerting	❌ Not implemented	No PagerDuty, OpsGenie, or similar

19.2 Recommended Additions

[!NOTE] **Proposed / Recommended**

- **Structured JSON logging** — Use `pino` or `winston` with JSON format for log aggregation
- **Request logging middleware** — Log method, path, status, duration for every request
- **Sentry/Datadog** — Error tracking and APM
- **Prometheus + Grafana** — Metrics and dashboards
- **CloudWatch** (if on AWS) — Log aggregation and alarms

20. Current Architecture Limitations

#	Limitation	Impact	Severity
1	In-memory OTP storage — OTPs lost on restart, not shareable across instances	Authentication breaks on restart or horizontal scaling	🔴 Critical
2	No rate limiting — OTP and all endpoints are unprotected from abuse	Brute-force attacks, API abuse	🔴 Critical
3	No SMS provider — OTPs are generated but never sent to phones	Users can't actually receive OTPs in production	🔴 Critical
4	Most features are client-side mock — Search, interests, messaging, notifications, payments, settings all use localStorage	These features don't persist across devices/sessions	🟡 High
5	6 sequential DB queries per profile view — No joins or batching	Performance degradation under load	🟡 High
6	No Docker configuration — Infrastructure directory is empty	Can't containerize for deployment	🟡 Medium
7	No CI/CD pipeline — No GitHub Actions, no automated testing	Manual deployment, no quality gates	🟡 Medium
8	Profile <code>:id</code> is public — No authorization on viewing other profiles	Any UUID can be used to access profile data	🟡 Medium
9	No CDN — CloudFront is configured but URL is empty	Photos served directly from S3 (higher latency)	🟢 Low
10	No database backups — No automated backup strategy	Data loss risk	🟡 Medium

21. Recommended Improvements

Priority	Improvement	Effort	Impact
🔴 P0	Integrate SMS provider (MSG91/Twilio) for real OTP delivery	2-3 days	Unblocks production launch
🔴 P0	Replace in-memory OTP Map with Redis	1-2 days	Enables horizontal scaling
🔴 P0	Add rate limiting (<code>@nestjs/throttler</code>) on auth endpoints	1 day	Prevents brute-force
🟡 P1	Implement search/discover backend (SearchModule)	3-5 days	Replace mock matches data
🟡 P1	Implement interests backend (InterestsModule)	3-4 days	Replace localStorage mock
🟡 P1	Add Docker + docker-compose for local + prod	2-3 days	Enables containerized deployment
🟡 P1	Implement payments (Razorpay/PhonePe)	5-7 days	Enable monetization
🟡 P1	Add CI/CD (GitHub Actions)	1-2 days	Automated testing + deployment
🟡 P2	Optimize profile queries (joins)	1-2 days	Improve read performance
🟡 P2	Configure CloudFront CDN	1 day	Improve photo delivery speed
🟡 P2	Implement notifications backend	3-4 days	Real-time user engagement
🟢 P3	Add structured logging (pino/winston)	1 day	Observability
🟢 P3	Add request logging middleware	0.5 day	Debugging
🟢 P3	Implement messaging backend	5-7 days	Post-match communication

22. Technology Decisions & Rationale

Decision	Choice	Rationale
Monorepo	PNPM workspaces + Turborepo	Shared types, validation, and database schema across frontend/backend; consistent builds
Frontend	Next.js 16 (App Router)	Server-side rendering for SEO (landing page), file-based routing, React 19
Backend	NestJS 11	Modular architecture (feature modules), built-in DI, Passport integration, TypeScript-first
ORM	Drizzle ORM	Type-safe SQL, lightweight, supports PostgreSQL natively, schema-as-code
Database	PostgreSQL	Relational data model fits matrimony profiles, strong JSONB support for arrays (preferences), ACID transactions
Validation	Zod (shared package)	Same schemas on frontend and backend; type inference; composable
Auth	Phone OTP + JWT	No passwords to manage; mobile-first UX; stateless tokens for scalability
File Storage	AWS S3 + presigned URLs	Direct browser-to-S3 uploads bypass API; reduces server load; bucket separation for sensitive docs
State Management	TanStack React Query	Server state caching, automatic refetching, mutation handling; avoids Redux complexity
UI Components	Radix UI + shadcn pattern	Accessible primitives, unstyled (customizable with Tailwind)
Styling	TailwindCSS 4 + CSS variables	Utility-first, custom design tokens for South Indian aesthetic

23. Appendix: Important Files and Their Responsibilities

Monorepo Root

File	Purpose
<code>package.json</code>	Root workspace scripts (<code>dev</code> , <code>build</code> , <code>db:*</code>)
<code>pnpm-workspace.yaml</code>	Workspace package locations (<code>apps/*</code> , <code>packages/*</code>)
<code>turbo.json</code>	Turborepo task configuration
<code>.env</code>	Environment variables (DB URL, JWT secret, AWS config, API URL)

Backend (`apps/api`)

File	Purpose
<code>main.ts</code>	Bootstrap: CORS, global prefix <code>/api</code> , exception filter, port 4000
<code>app.module.ts</code>	Root module: imports Config, Database, Health, Auth, Profiles, Preferences, Media
<code>auth/auth.controller.ts</code>	<code>POST send-otp</code> , <code>POST verify-otp</code> , <code>GET me</code>
<code>auth/auth.service.ts</code>	OTP generation/verification, user creation, JWT signing
<code>auth/jwt.strategy.ts</code>	Passport JWT strategy — token validation + user lookup
<code>profiles/profiles.controller.ts</code>	<code>POST complete-registration</code> , <code>GET me</code> , <code>GET :id</code>
<code>profiles/profiles.service.ts</code>	Multi-table transactional profile creation, profile assembly
<code>media/media.controller.ts</code>	Presigned upload URL, confirm photo/verification/horoscope, delete photo
<code>media/media.service.ts</code>	Media business logic, profile ownership verification
<code>media/providers/s3.provider.ts</code>	AWS S3 presigned URL generation, delete, mock mode for local dev
<code>preferences/preferences.controller.ts</code>	<code>GET me</code> , <code>PUT me</code> for partner preferences
<code>preferences/preferences.service.ts</code>	Preferences retrieval and upsert
<code>health/health.controller.ts</code>	<code>GET /api/health</code> — DB connectivity check
<code>common/guards/auth.guard.ts</code>	JWT authentication guard (Passport wrapper)
<code>common/pipes/zod-validation.pipe.ts</code>	Zod schema validation pipe with field-level error formatting
<code>common/filters/http-exception.filter.ts</code>	Global exception filter — normalizes all errors to JSON
<code>common/decorators/current-user.decorator.ts</code>	Extracts <code>UserSession</code> from <code>request.user</code>
<code>config/</code>	Namespaced config: <code>app</code> , <code>auth</code> , <code>database</code> , <code>storage</code>
<code>database/database.module.ts</code>	Global Drizzle DB client provider

Frontend (`apps/web`)

File	Purpose
<code>app/layout.tsx</code>	Root layout: Google Fonts (Mukta, Cormorant Garamond, Tiro Tamil), QueryProvider
<code>app/(public)/page.tsx</code>	Landing page (31KB — hero, features, testimonials)
<code>app/(dashboard)/layout.tsx</code>	Dashboard layout wrapper → <code>DashboardShell</code>
<code>lib/api-client.ts</code>	Singleton <code>ApiClient</code> — fetch wrapper, JWT management, namespaced API methods
<code>lib/profile-store.ts</code>	<code>SignupData</code> type, <code>saveProfile()</code> , <code>loadProfile()</code> (sessionStorage)
<code>lib/validation.ts</code>	Frontend-only Zod schemas for wizard steps, profile edit, search, checkout
<code>lib/matches.ts</code>	Hardcoded mock match profiles (6 profiles)
<code>lib/user-activity.ts</code>	Client-side mock: interests, shortlists, notifications, settings (all localStorage)
<code>lib/plans.ts</code>	Membership plan definitions, mock subscription/invoice storage
<code>lib/portal-access.ts</code>	Profile completeness check, verification gate, mock dashboard data
<code>hooks/queries.ts</code>	All React Query hooks (queries + mutations)
<code>providers/query-provider.tsx</code>	TanStack React Query client provider
<code>components/layout/dashboard-shell.tsx</code>	Authenticated shell: top nav, profile avatar, notifications, mobile bottom nav

Shared Packages

File	Purpose
<code>packages/database/src/client.ts</code>	<code>createDbClient()</code> — postgres.js + Drizzle ORM factory
<code>packages/database/src/schema/</code>	8 table schemas: users, profiles, family_details, lifestyle_interests, horoscopes, partner_preferences, profile_photos, verifications
<code>packages/database/drizzle.config.ts</code>	Drizzle Kit configuration for migrations
<code>packages/types/src/</code>	Shared TypeScript interfaces: User, Profile, AuthResponse, FullProfileView, media types
<code>packages/validation/src/</code>	Shared Zod schemas: auth, profile, preferences, media